

Runtime Scaling Analysis Report

Created with results *from N = 1 to N = 32*

Executive Summary

This report analyzes computation-time measurements for $N = 1$ through $N = 32$. The runtime increases from 1.56 seconds at $N = 1$ to 849.1 seconds at $N = 32$, indicating strong nonlinear growth.

Cubic and quadratic curves fit the finite $N = 1$ to 32 window better than a single full-range exponential curve. However, the semi-log plot still shows that runtime growth is strongly multiplicative, especially in the larger- N tail. Therefore, this report separates two ideas: finite-window curve fitting and scalability interpretation.

A full-range exponential fit gives $t(N) \approx 3.403451 \exp(0.193680 N)$, corresponding to an average multiplier of $\exp(0.193680) \approx 1.214$; but this single exponential overestimates later growth because early small- N runtimes grow very quickly. A tail-only exponential fit on $N \geq 17$ gives $t(N) \approx 31.242948 \exp(0.102762 N)$, corresponding to a more conservative later-stage multiplier of $\exp(0.102762) \approx 1.108$.

1. Complete Dataset

The table below lists all measurements used in the updated analysis.

N	Time (s)	N	Time (s)
1	1.56	17	181.9
2	2.11	18	186.9
3	3.23	19	213.1
4	4.10	20	247.1
5	6.86	21	275.6
6	9.24	22	308.1
7	15.64	23	370.3
8	17.07	24	379.9
9	29.94	25	442.8
10	30.27	26	398.0
11	54.97	27	470.9
12	47.74	28	515.4
13	79.61	29	597.8
14	72.51	30	668.8
15	112.50	31	828.8
16	92.58	32	849.1

2. Visual Runtime Trend

Figure 1 shows the observed computation times together with the full-range exponential fit. The curve captures the overall nonlinear increase, while local deviations show possible instance-dependent effects.

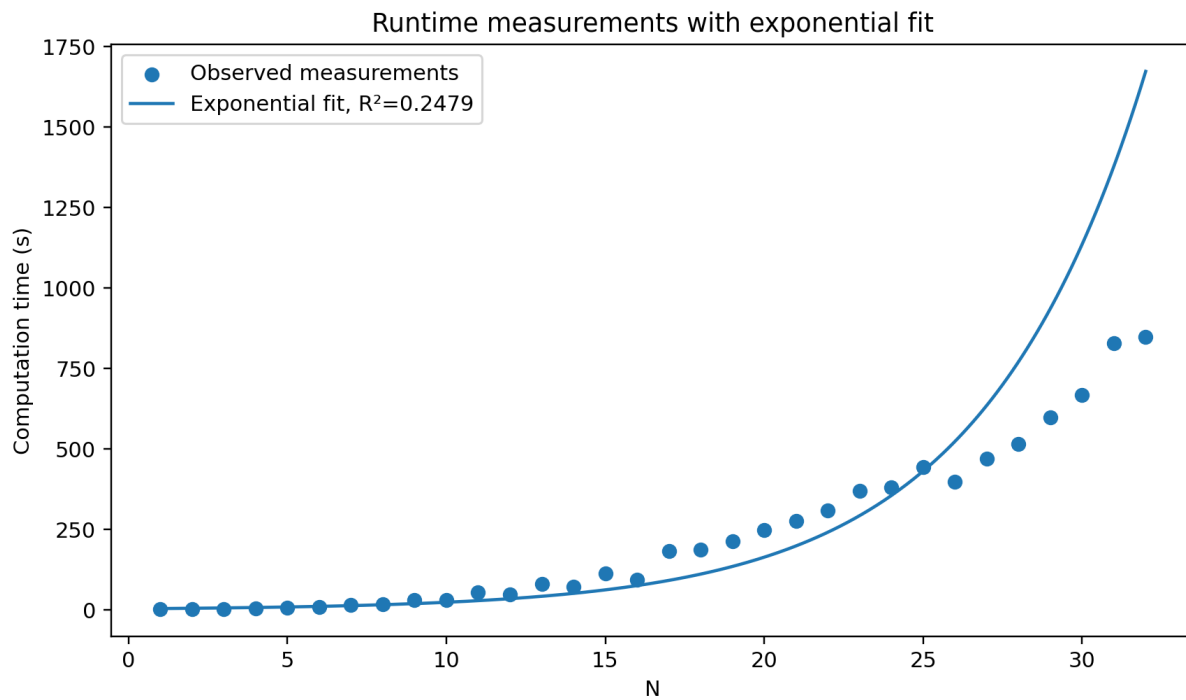


Figure 1. Runtime measurements and exponential fit for $N = 1$ to $N = 32$.

3. Model Comparison

Several simple models were compared: linear, quadratic, cubic, exponential, and power-law. The goal is not to prove the exact theoretical complexity, but to identify which simple empirical model best describes the observed data.

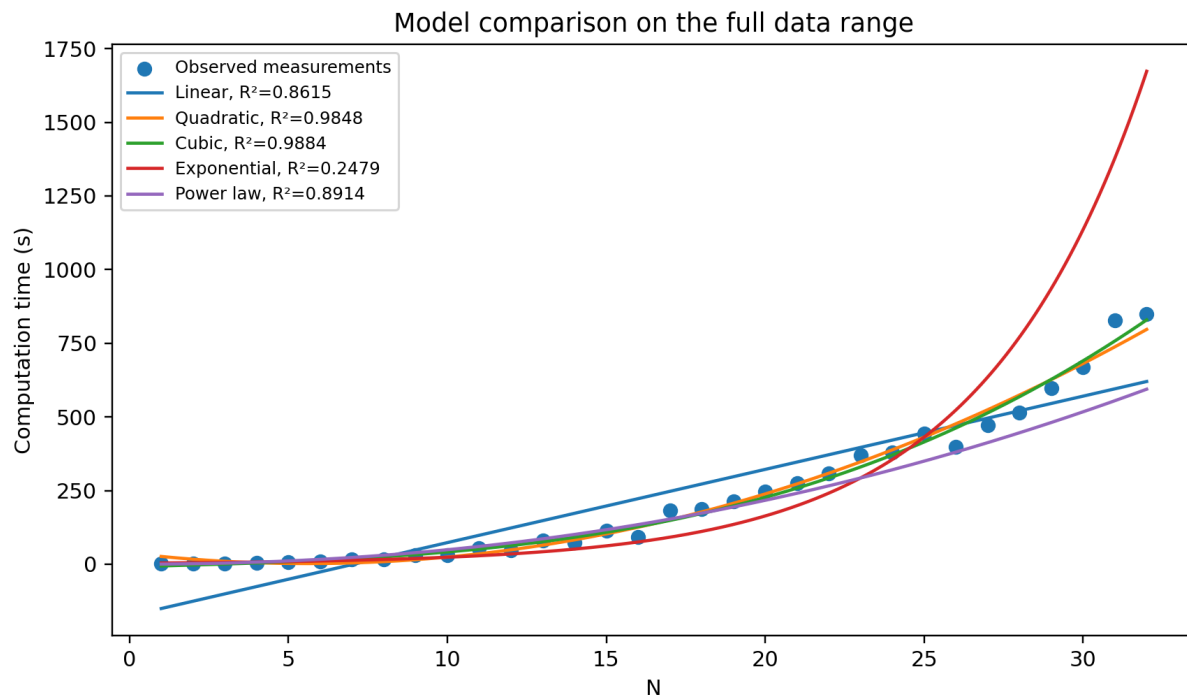


Figure 2. Model comparison on the full observed range.

The model-quality metrics are shown below on the raw second-scale. R^2 measures explained variance, RMSE gives the typical error in seconds, and MAPE gives the average percentage error. Because small- N runtimes are close to zero, percentage errors can look large even when absolute errors are moderate.

Model	R^2	RMSE (s)	MAPE (%)
Cubic	0.9884	26.66	38.38
Quadratic	0.9848	30.47	95.71
Power law	0.8914	81.51	30.12
Linear	0.8615	92.03	736.02
Exponential	0.2479	214.48	45.58

Full-range model equations:

Linear: $t(N) = 24.8620 N + -175.3969$

Quadratic: $t(N) = 1.1406 N^2 + -12.7781 N + 37.8970$

Cubic: $t(N) = 0.0240 N^3 + -0.0463 N^2 + 3.1317 N + -9.1849$

Exponential: $t(N) = 3.403451 \exp(0.193680 N)$

Power law: $t(N) = 0.351789 N^{2.144333}$

4. Semi-log Analysis

A semi-log plot is useful for detecting exponential behavior. When an exponential relation $t(N) = A \exp(BN)$ is plausible, the points should appear approximately linear after applying a logarithmic scale to the y-axis.

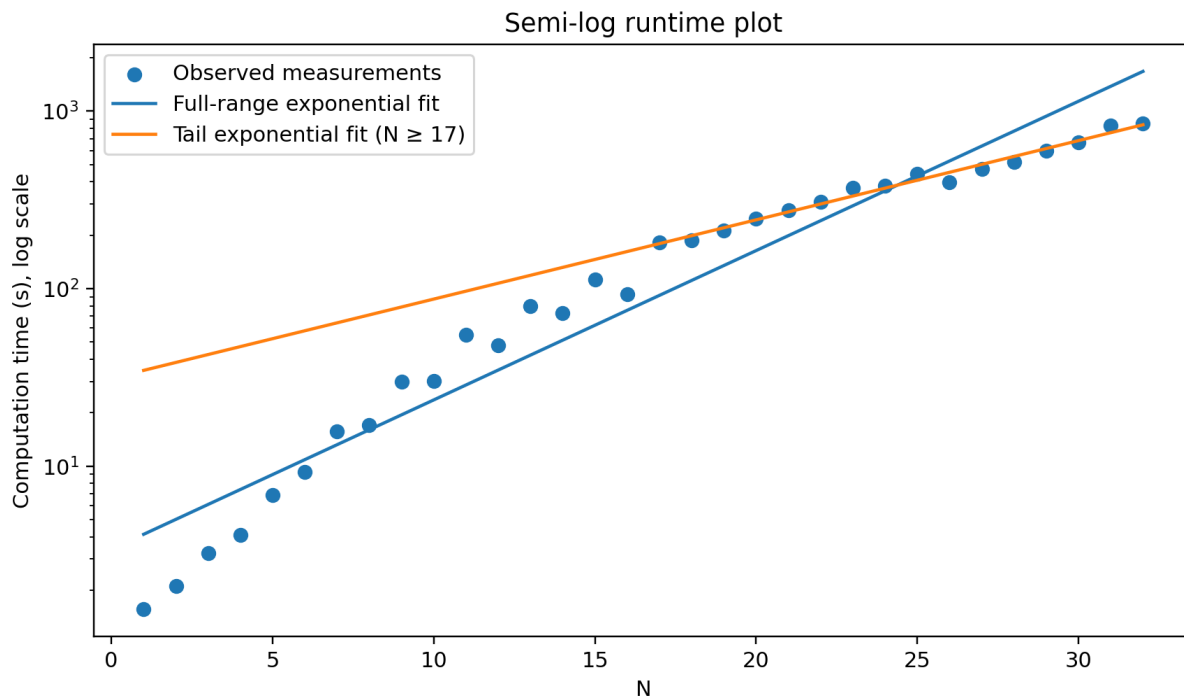


Figure 3. Semi-log runtime plot with full-range and tail exponential fits.

The semi-log plot supports a multiplicative-growth interpretation, but not a single perfect exponential law over the entire range. The full-range exponential fit has log-space $R^2 = 0.9348$ with $B = 0.193680$. The tail-only fit for $N \geq 17$ has log-space $R^2 = 0.9832$ with $B = 0.102762$. The tail growth rate is lower because the early measurements grow rapidly relative to their small starting values.

5. Noise, Deviations, and Growth Ratios

The data are not perfectly smooth. For example, $N = 12$ is lower than $N = 11$, $N = 14$ is lower than $N = 13$, $N = 16$ is lower than $N = 15$, and $N = 26$ is lower than $N = 25$.

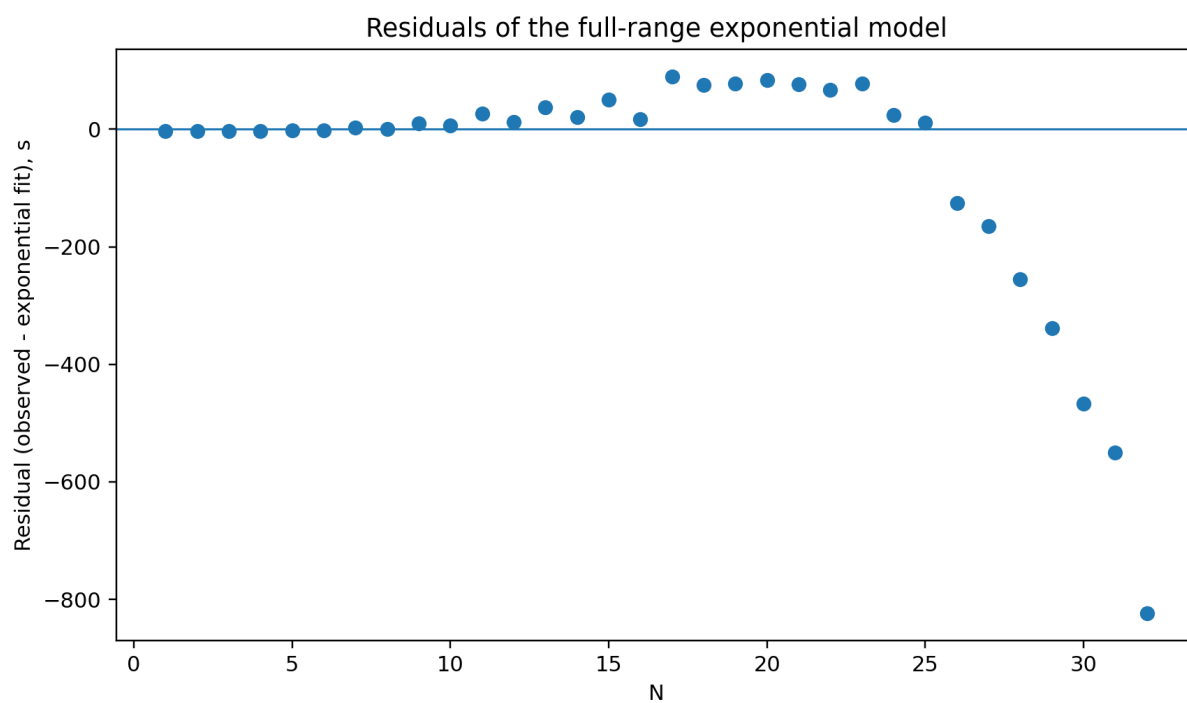


Figure 4. Residuals for the full-range exponential fit.

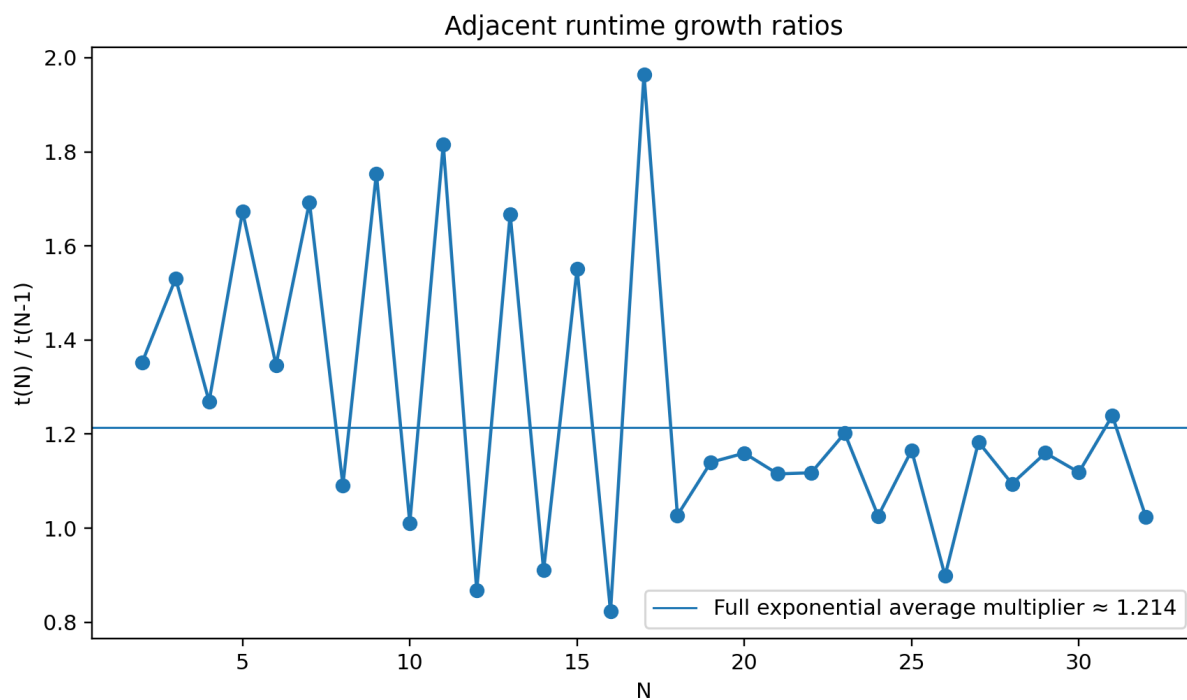


Figure 5. Adjacent runtime ratios $t(N)/t(N-1)$.

The adjacent ratio plot shows that the per-step growth multiplier is not constant. This is expected in empirical runtime measurements, especially when the solver has stochastic or instance-dependent behavior.

6. Extrapolation to $N = 100$

Using the full-range exponential fit, the predicted runtime for $N = 100$ is approximately 877,672,328 seconds, which is about 243,797.9 hours or 10,158.2 days. This might be too pessimistic because the full-range fit is pulled upward by the rapid relative growth at very small N .

Using only the tail data $N \geq 17$ gives a much lower prediction: approximately 907,049 seconds, which is about 252.0 hours or 10.5 days. The large gap between the two estimates illustrates why extrapolating from $N \leq 32$ to $N = 100$ is highly uncertain.

Therefore, the $N = 100$ prediction should be treated as an order-of-magnitude scenario rather than a reliable forecast. The true runtime may be much smaller or much larger depending on solver behavior and theoretical complexity.

7. Interpretation and Practical Implications

The measurements alone do not prove that the algorithm has exact exponential theoretical complexity. They show that the observed implementation behaves approximately exponentially over the tested range. To make a stronger complexity claim, the empirical results should be paired with an analysis of the solver formulation.

8. Conclusion

The overall trend is clearly nonlinear. The best raw-scale finite-window fits are polynomial, but the semi-log plot and the $N \geq 17$ tail fit support a multiplicative-growth interpretation for scalability discussion. The data also show measurement noise and local nonmonotonicity, so all long-range extrapolations, especially to $N = 100$, should be presented cautiously.

Appendix: Model Code

The following Python code reproduces the numerical analysis used in this report. It defines the full $N=1$ to $N=32$ runtime dataset, fits linear, quadratic, and exponential models, computes R^2 scores, predicts runtime at $N=100$, and generates the supporting plots.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Full runtime dataset, in seconds
N = np.arange(1, 33, dtype=float)
time_s = np.array([
    1.56, 2.11, 3.23, 4.10, 6.86, 9.24, 15.64, 17.07,
    29.94, 30.27, 54.97, 47.74, 79.61, 72.51, 112.50, 92.58,
    181.90, 186.90, 213.10, 247.10, 275.60, 308.10, 370.30, 379.90,
    442.80, 398.00, 470.90, 515.40, 597.80, 668.80, 828.80, 849.10
])

# Store the raw data in a dataframe
runtime_df = pd.DataFrame({"N": N.astype(int), "time_s": time_s})

# Helper function for coefficient of determination
def r2_score(y_true, y_pred):
    residual_sum = np.sum((y_true - y_pred) ** 2)
    total_sum = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1.0 - residual_sum / total_sum

# 1. Linear model:  $t(N) = aN + b$ 
linear_coef = np.polyfit(N, time_s, deg=1)
linear_pred = np.polyval(linear_coef, N)

# 2. Quadratic model:  $t(N) = aN^2 + bN + c$ 
quadratic_coef = np.polyfit(N, time_s, deg=2)
quadratic_pred = np.polyval(quadratic_coef, N)

# 3. Exponential model:  $t(N) = A \exp(BN)$ 
# Fit in log-space:  $\log(t) = \log(A) + BN$ 
exp_log_coef = np.polyfit(N, np.log(time_s), deg=1)
B = exp_log_coef[0]
A = np.exp(exp_log_coef[1])
exponential_pred = A * np.exp(B * N)

# Compare model quality
model_results = pd.DataFrame({
    "Model": ["Linear", "Quadratic", "Exponential"],
    "R2": [
        r2_score(time_s, linear_pred),
        r2_score(time_s, quadratic_pred),
        r2_score(time_s, exponential_pred)
    ]
})

print(model_results)
print(f"Linear:       $t(N) = \{linear\_coef[0]:.6f\} N + \{linear\_coef[1]:.6f\}"$ )
print(f"Quadratic:    $t(N) = \{quadratic\_coef[0]:.6f\} N^2 + \{quadratic\_coef[1]:.6f\} N + \{quadratic\_coef[2]:.6f\}"$ )
print(f"Exponential:  $t(N) = \{A:.6f\} * \exp(\{B:.6f\} N)"$ )

# Extrapolated prediction for  $N=100$ 
pred_N = 100
pred_time_s = A * np.exp(B * pred_N)
pred_time_h = pred_time_s / 3600
pred_time_d = pred_time_h / 24

print(f"Predicted time for  $N=\{pred\_N\}$ :  $\{pred\_time\_s:.2f\}$  seconds")
print(f"Predicted time for  $N=\{pred\_N\}$ :  $\{pred\_time\_h:.2f\}$  hours")
print(f"Predicted time for  $N=\{pred\_N\}$ :  $\{pred\_time\_d:.2f\}$  days")

# Smooth curve for plotting fitted models
N_smooth = np.linspace(N.min(), N.max(), 400)
linear_smooth = np.polyval(linear_coef, N_smooth)
quadratic_smooth = np.polyval(quadratic_coef, N_smooth)
exponential_smooth = A * np.exp(B * N_smooth)

# Plot 1: raw runtime curve
plt.figure(figsize=(8, 5))
plt.scatter(N, time_s, label="Observed runtime")
plt.plot(N, time_s, alpha=0.6)
plt.xlabel("N")
```

```

plt.ylabel("Runtime (seconds)")
plt.title("Observed Runtime from N=1 to N=32")
plt.legend()
plt.tight_layout()
plt.savefig("runtime_observed.png", dpi=200)
plt.close()

# Plot 2: model comparison
plt.figure(figsize=(8, 5))
plt.scatter(N, time_s, label="Observed runtime")
plt.plot(N_smooth, linear_smooth, label="Linear fit")
plt.plot(N_smooth, quadratic_smooth, label="Quadratic fit")
plt.plot(N_smooth, exponential_smooth, label="Exponential fit")
plt.xlabel("N")
plt.ylabel("Runtime (seconds)")
plt.title("Runtime Model Comparison")
plt.legend()
plt.tight_layout()
plt.savefig("runtime_model_comparison.png", dpi=200)
plt.close()

# Plot 3: semi-log plot
plt.figure(figsize=(8, 5))
plt.scatter(N, time_s, label="Observed runtime")
plt.plot(N_smooth, exponential_smooth, label="Exponential fit")
plt.yscale("log")
plt.xlabel("N")
plt.ylabel("Runtime (seconds, log scale)")
plt.title("Semi-log Runtime Plot")
plt.legend()
plt.tight_layout()
plt.savefig("runtime_semilog.png", dpi=200)
plt.close()

```